

Building a Real-Time Payment Gateway Integration with PHP, NGROK, PayHere, and MySQL



This guide demonstrates how to build a full PHP payment gateway integration using PayHere Sandbox, ngrok, and MySQL, covering setup, configuration, database initialization, secure callbacks, transaction verification, email notifications, and receipt generation.

Article

Access the Article from any of the URLs below

- LinkedIn: <https://www.linkedin.com/pulse/building-real-time-payment-gateway-integration-php-liyanarachchi-yuigc/>
- Medium: <https://dulannithilaliyanarachchi.medium.com/building-a-real-time-payment-gateway-integration-with-php-payhere-ngrok-and-mysql-533c5e4a5720>
- AWS: <https://builder.aws.com/content/3ImAUjqYN0gmihHAXeIIVJoHFn/building-a-real-time-payment-gateway-integration-with-php-payhere-ngrok-and-mysql>
- DEV Community: <https://dev.to/dulannithilaliyanarachchi/building-a-real-time-payment-gateway-integration-with-php-payhere-ngrok-and-mysql-dnf>

- CODERLEGION: <https://coderlegion.com/26585/building-a-real-time-payment-gateway-integration-with-php-payhere-ngrok-and-mysql>

Requirements

- PHP 8.0 or later
- MySQL Database Server
- Apache Server (XAMPP, WAMP, or MAMP)
- Composer

Complete Guide

ngrok Setup

ngrok is used to expose your local PHP application to the internet so PayHere can reach the endpoints specially the notify_url.

1. Go to <https://ngrok.com/>
2. Install ngrok and sign in to your ngrok account.
3. Add your authtoken to the ngrok configuration using the following command

```
ngrok config add-authtoken YOUR_NGROK_AUTHTOKEN
```

4. Start the tunnel for the port where your PHP server is running.(localhost port) using the following command.

```
ngrok http 80
```

If your app runs on a different port, replace 80 with that port number.

5. Copy the public HTTPS URL shown by ngrok
6. To Stop the ngrok tunnel running in the terminal press CTRL + C

PayHere Sandbox Setup

Go to PayHere Sandbox Signup Page at <https://sandbox.payhere.lk/merchant/sign-up> and create an account, then verify the email by entering the verification code sent to that inbox.

1. Sign in to the sandbox dashboard after the account is created.
2. Enter the profile details such as First Name, Last Name, Phone Number, and Business Name.
3. Go to Settings and create a new API key or domain/app credential set, depending on the dashboard view shown in your account.
 - Enter the app name as any label you want.
 - Add the domain of the website or mobile app that will be hosted which will be the full ngrok domain(wildcardsubdomain.maindomain.domaintype) of the copied public HTTPS ngrok URL in the setup process.
 - Turn on the first permission toggle from the permissions list if it is required by the dashboard.
4. Go to Integrations and add a Domain or App.
 - Select Domain for a website or App for a mobile application.
 - Brand Name will usually auto-fill from the business name, but you can change it if needed.
 - Add the maindomain.domaintype(ngrok-free.dev) as the domain.
 - Copy the Merchant Secret and keep it safe. In this project, the secret is used to generate the payment hash.
 - Copy the Merchant ID as well. It is required in the checkout form.

Important: for the Checkout API used by this project, the Merchant Secret is tied to the exact domain or app that is registered in PayHere. If you change the tunnel URL or move the project to another domain, add that new domain/app again and get a new Merchant Secret.

For Additional Information Provided by Payhere for the Sandbox Setup and Configuration See <https://support.payhere.lk/>

Source Code Overview

Access the source code from the GitHub repository below:

<https://github.com/NithilaLD/Building-a-Real-Time-Payment-Gateway-Integration-with-PHP-PayHere-ngrok-and-MySQL/>

File	Purpose
index.php	Shows the payment form where the customer enters name, email, phone, and amount.
checkout.php	Builds the PayHere checkout payload, generates the hash, and redirects PayHere.
notify.php	Receives PayHere server callback data and writes the payment payload to payment-log.txt.
success.php	Verifies the payment, saves it into the database, sends email, and can generate a receipt PDF.
cancel.php	Shows the cancelled payment screen.
config.php	Stores Merchant ID, Merchant Secret, database details, and SMTP settings.
payment_gateway.sql	Creates the database and the test table used by the application.

Copy the public HTTPS URL shown by ngrok and use it in the callback URLs inside checkout.php. If the project is inside a subfolder such as /payment/, keep that path in the return, cancel, and notify URLs. See the sample below.

```
<input type="hidden" name="return_url" value="https://YOUR-NGROK-Full-DOMAIN/payment/success.php">
<input type="hidden" name="cancel_url" value="https://YOUR-NGROK-Full-DOMAIN/payment/cancel.php">
<input type="hidden" name="notify_url" value="https://YOUR-NGROK-Full-DOMAIN/payment/notify.php">
```

Database Setup

The database file supplied with the project creates a database named `payment_gateway` and a table named `test`.

1. Start MySQL from XAMPP, WAMP, MAMP, or your local database service.
2. Open phpMyAdmin and create a database named `payment_gateway` if it does not already exist.
3. Import the `payment_gateway.sql` file into that database.
4. Confirm that the `test` table exists and that the imported columns match the application code.
5. The table is used to store successful payment records and to prevent duplicate inserts.
6. The full db creation code is there in the GitHub repository mentioned in source code section.

Column	Purpose
<code>t_id</code>	Auto-increment primary key.
<code>email_flag</code>	Marks whether the confirmation email has already been sent.
<code>order_id</code>	Unique order reference from the checkout page.
<code>name</code>	Customer name.
<code>email</code>	Customer email address.
<code>phone</code>	Customer phone number.
<code>amount</code>	Paid amount.
<code>currency</code>	Payment currency, usually LKR.
<code>payment_id</code>	PayHere transaction ID.
<code>payment_status</code>	Stores PAID after verification.
<code>paid_at</code>	Timestamp for the successful transaction.

Update the PHP Configuration

The project uses config.php to store the PayHere credentials, database details, and SMTP values. Replace every placeholder with your own values.

```
return [  
    'merchant_id' => 'YOUR_MERCHANT_ID',  
    'merchant_secret' => 'YOUR_MERCHANT_SECRET',  
  
    'db' => [  
        'host' => 'YOUR_DB_HOST',  
        'user' => 'YOUR_DB_USER',  
        'pass' => 'YOUR_DB_PASSWORD',  
        'name' => 'payment_gateway',  
    ],  
  
    'smtp' => [  
        'host' => 'YOUR_EMAIL_SMTP_HOST',  
        'username' => 'YOUR_EMAIL_ADDRESS',  
        'password' => 'YOUR_EMAIL_PASSWORD_OR_APP_PASSWORD',  
        'port' => 465,  
        'from_email' => 'YOUR_EMAIL_ADDRESS',  
        'from_name' => 'Your App Name',  
    ],  
];
```

Install Project Dependencies

1. Open a terminal in the project folder.
2. Create a new file named **composer.json** in the root directory of the project and add the following code.

```
{
  "require": {
    "phpmailer/phpmailer": "^7.1",
    "mpdf/mpdf": "^8.3"
  }
}
```

3. Save the file.
4. Run the command below in the terminal to download and install the required packages.

```
composer install
```

5. After the installation is completed, confirm that the ``vendor`` folder and the ``vendor/autoload.php`` file are created successfully before testing the success page.

Running the Application

1. Place the project in your local web server folder, such as `htdocs/payment` or `www/payment`.
2. Start Apache and MySQL.
3. Start ngrok and copy the public URL.
4. Open `index.php` in the browser through the ngrok URL.
5. Submit the form with a test name, email, phone number, and amount.

1. The payment flow works like this:

- `index.php` collects the customer details.
- `checkout.php` formats the amount, creates the `order_id`, generates the PayHere hash, and posts the customer to the sandbox checkout page.
- PayHere sandbox processes the card and sends the status back through `return_url`, `cancel_url`, and `notify_url`.
- `notify.php` saves the raw callback payload into `payment-log.txt`.

- success.php verifies the hash, stores the transaction in the database, sends the confirmation email, and offers a PDF receipt download.

Troubleshooting :

- 404 on success.php or cancel.php usually means the ngrok URL or subfolder path is wrong.
- If PayHere does not return to your site, confirm that notify_url is public and reachable over HTTPS.
- If the payment succeeds but no row is saved, verify the MySQL credentials in config.php and confirm that the test table exists.
- If the receipt email fails, check your SMTP host, username, password, and port.
- If duplicate records appear, the code checks order_id to avoid inserting the same payment twice.

Sandbox Test Cards

PayHere sandbox does not process real money. The official sandbox test card set includes Visa, MasterCard, and AMEX numbers. Any other card number should fail. Use the sample cards below for successful test payments.

Card Type	Test Card Number
Visa	4916217501611292
MasterCard	5307732125531191
AMEX	346781005510225

For Name on Card, CVV, and expiry date, enter any valid data that the sandbox accepts.

For Additionally to test insufficient funds, card expired, etc. like scenarios there are multiple more dummy card numbers are provided in <https://support.payhere.lk/sandbox-and-testing>

Final reminder: use only the sandbox environment and test cards while developing. Switch to live credentials only after the full flow works end to end.